Probléma-megoldási stratégiák:

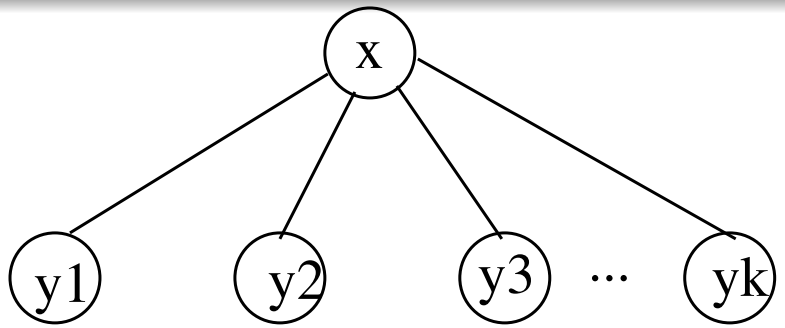
Visszalépéses keresés
Keresés elágazás-korlátozással

Szerző: Horváth Gyula
Előadó:Németh Zsolt

2025. november 18.

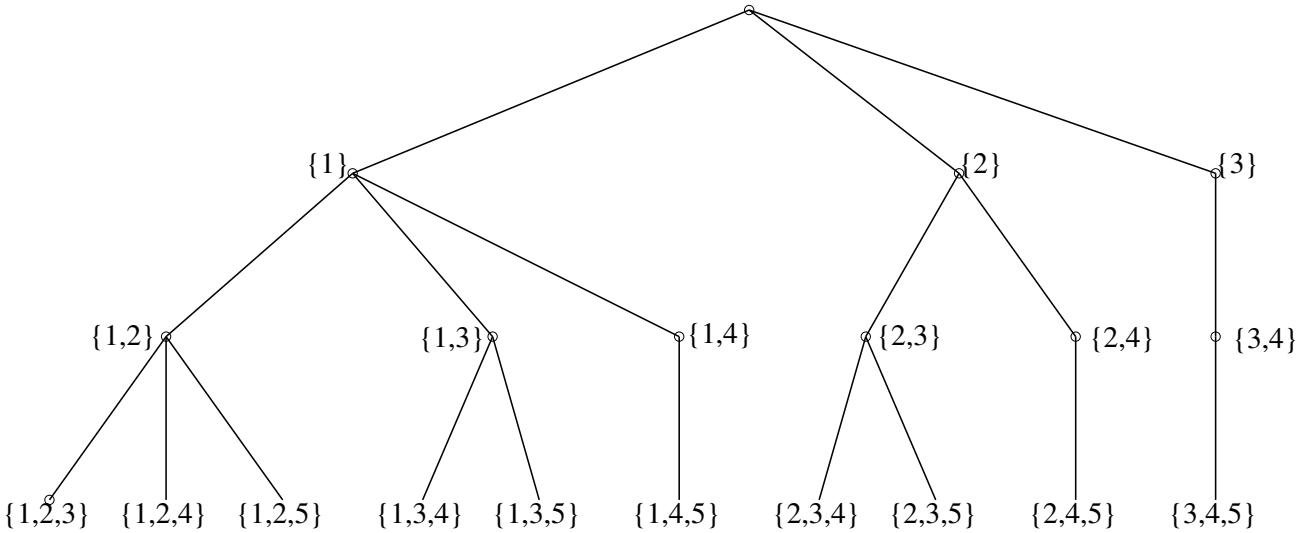
Megoldás keresése visszalépéssel (Backtracking)

A módszer olyan problémák megoldására alkalmazható, ahol a megoldás kifejezhető $X = \langle x_1, \ldots, x_k \rangle$ sorozat formájában. A lehetséges X megoldások kezdőszeletei , a **megoldáskezdemények** egy fát alkotnak, ezt nevezzük a probléma **megoldásterének**. Ellentétben a kimerítő kereséssel, nem csak a lehetséges megoldásokat vesszük sorba, hanem a megoldáskezdeményeket is. Ez lehetővé teszi, hogy ha egy megoldáskezdeményről kiderül, hogy semmilyen folytatása nem lehet megoldása a problémának, akkor az összes folytatását kizárjuk a keresésből.
A megoldáskezdemények gondolatilag alkotnak fát, csak azokat a fapontokat tároljuk megfelelő adatszerkezettel, amelyekre szükség van a keresés adott állapotában. Ha az x megoldáskezdeménynek $y_1, \ldots, y_k$ a közvetlen folytatásai, akkor a megoldástér fájában $y_1, \ldots$ és y_k gyermekei lesznek az x-et reprezentáló fapontnak.



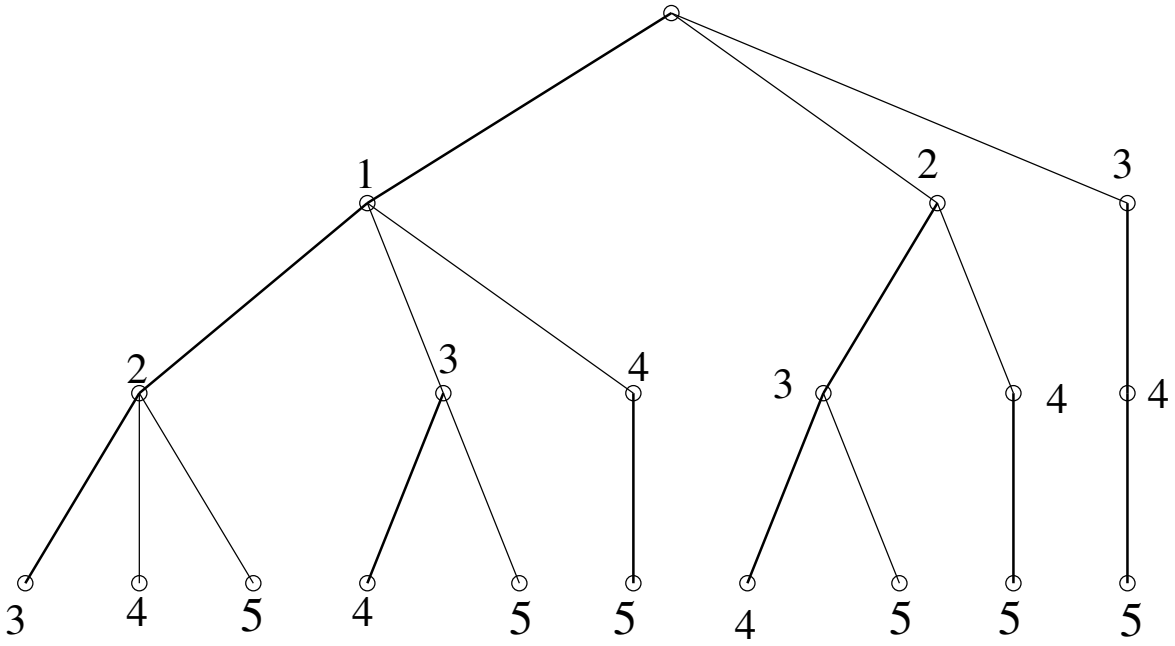
1. ábra

Például, ha a megoldandó probléma megoldása kifejezhető (5, 3) kombinációként, akkor a megoldástér ábrázolható az alábbi fával, ahol



2. ábra. (5,3) kombinációk fája

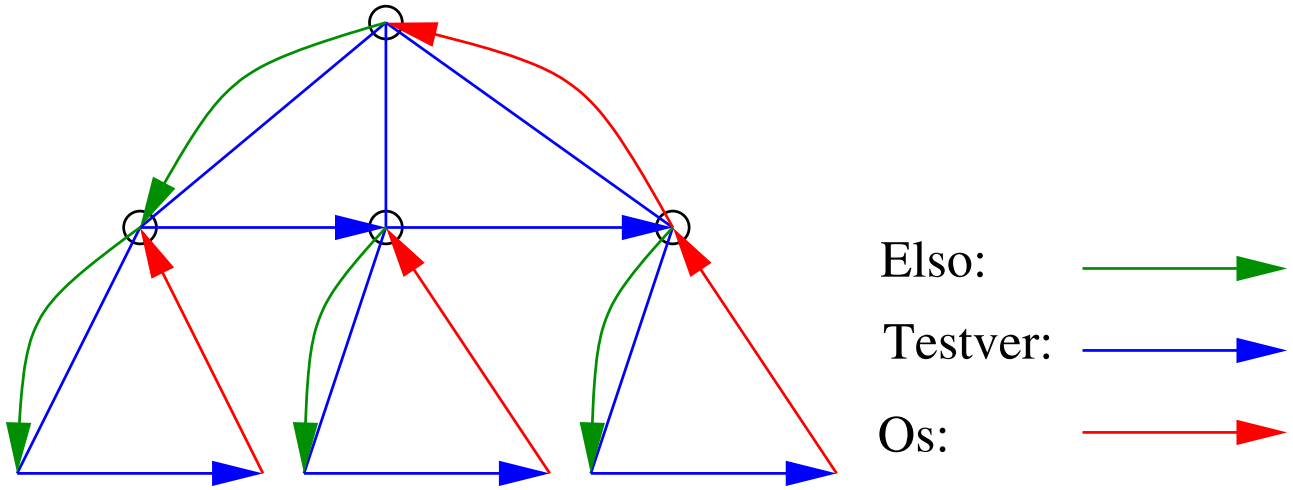
Gyakran nem a kezdőszeleteket jelöljük a fa pontjaként, hanem a kezdőszeletet a gyökértől az adott pontig vezető úton lévő adatok sorozata adja. Et jobban kifejezi azt, hogy a megoldáskezdeményeket lépésről lépésre építjük a bejárás során.



3. ábra

Tehát minden fabejáró algoritmus alkalmazható megoldás keresésére és a bejárás során alkalmazhatjuk a kizárást. Ahhoz, hogy a megoldástere mint fát bejárjuk rekurzív algoritmussal, elegendő két művelet. Ha az x megoldáskezdemény közvetlen

lehetséges folytatásai y_1, ... , y_k, akkor kell egy művelet, amely x-ből előállítja az első folytatást, azaz y_1-et, és kell egy olyan művelet, amely y_i-ből előállítja y_{i+1}-et. Persze azt is jeleznie kell, ha nincs következő testvért.
Ha iteratívan akarjuk bejárni a megoldásteret, akkor kell még egy olyan művelet, amely y_k-ból előállítja x-et.



4. ábra. Fa bejáráshoz alkalmazandő műveletek

Alapműveletek fák bejárásához

`bool Elso(FaPont& X)`

Ha létezik az X fapontnak első gyereke, akkor X értéke az első gyerek lesz és igaz értéket ad vissza, egyébként X értéke nem változik és hamis értéket ad vissza.

`bool Testver(FaPont& X)`

Ha létezik az X fapontnak következő testvére, akkor X értéke a következő testvér lesz és igaz értéket ad vissza, egyébként X értéke nem változik és hamis értéket ad vissza.

`bool Os(FaPont& X)`

Ha létezik az X fapontnak közvetlen őse, akkor X értéke a közvetlen ős lesz és igaz értéket ad vissza, egyébként X értéke nem változik és hamis értéket ad vissza.

Megjegyezzük, hogy az *Os* műveletet csak olyan fapontra kell megvalósítani, amelynek nincs testvére.

Fa rekurzív bejárása mentéssel

Mivel az **Elso(X)** művelet megváltoztatja X értékét, ezért a művelet előtt el kell menteni

```
1 void rFaBejar(FaPont X){
2   //rekurzív fabejárás
3   //az X fapont feldolgozása
4   X.Kilr();
5   if(!X.Elso()) return;
6   do{
7     rFaBejar(X);
8   }while(X.Testver());
9 }
```

Fa rekurzív bejárása visszaállítással

Ha megadjuk az Elso() műveket iElso(), illetve a Testver() művelet iTestver() inverzét, akkor a rekurzív bejárást ezekkel is megvalósíthatjuk. Az inverz műveltek visszaállítják a fapont művelet előtti értékét.

```
1  void iFaBejar(FaPont& X){
2    //az X fapont feldolgozása
3    if(Elso(X)){
4      iFaBejar(X);
5      iElso(X);
6    }
7    if(Testver(X)){
8      iFaBejar(X);
9      iTestver(X);
10   }
11 }
```

Fa iteratív bejárása

```
1  void FaBejar(FaPont& X){
2    //iteratív fa bejárás
3    bool tovabb=true;
4    bool elsore=true;
5    while(tovabb){
6      if(elsore){
7        //az X fapont feldolgozása
8        elsore=Elso(X);
9      }else{
10       if(Testver(X))
11         elsore=true;
12       else//nincs testvére, visszalépés
13         tovabb=Os(X);
14     }
15   }
16 }
```

A megoldás keresését meg tudjuk fogalmazni olyan általános formában, hogy az algoritmus érdemi része, azaz a megoldástér bejárása csak néhány problémaspecifikus műveletet alkalmaz.
Ezt módszert "application framework" módszernek is nevezik.
Adott problémára nem kell újraírni az érdemi részt, csak a problémaspecifikus műveletek megvalósítását kell megadni.
A fabejáró műveletek mellett még két probléma-specifikus művelet kell; a **Lehet()** kizáró művelet, és **Megoldas**(), amely teszteli, hogy az adott megoldáskezdemény megoldása-e a problémának.
bool X.Lehet();
Akkor és csak akkor adjon hamis értéket, ha biztosan nincs megoldás a megoldástér X-gyökerű részfájában. Ha igaz értéket ad, abból nem következik, hogy létezik is megoldás az X-gyökerű részfában.X értéke nem változhat.
Tehát hamisat ad, ha az X fapont által reprezentált megoldáskezdeménynek nincs olyan folytatása, amely megoldása a problémának.

bool X.Megoldas();
Akkor és csak akkor adjon igaz értéket, ha X fapont által reprezentált megoldáskezdemény megoldása a problémának. X értéke nem változhat.
A továbbiakban *FaPont* azonosítót használunk a megoldástér pontjainak típusára. Az *FaPont* adatszerkezet, amely tartalmazza a megoldáskezdeményt és olyan kiegészítő adatokat, amelyek segítik a műveletek (hatékony megvalósítását.

```
1  class FaPont{//a megoldástér pontja
2    //Probléma-specifikus adatoktagok
3    public:
4    bool Elso();
5    bool Testver();
6    bool Megoldas();
7    bool Lehet();
8  //   bool Os(); //nemrekurzív keresés esetén
9    FaPont(){//üres megoldáskezdeményt létesítő konst
10   }
11   void Kilr();
12 };
```

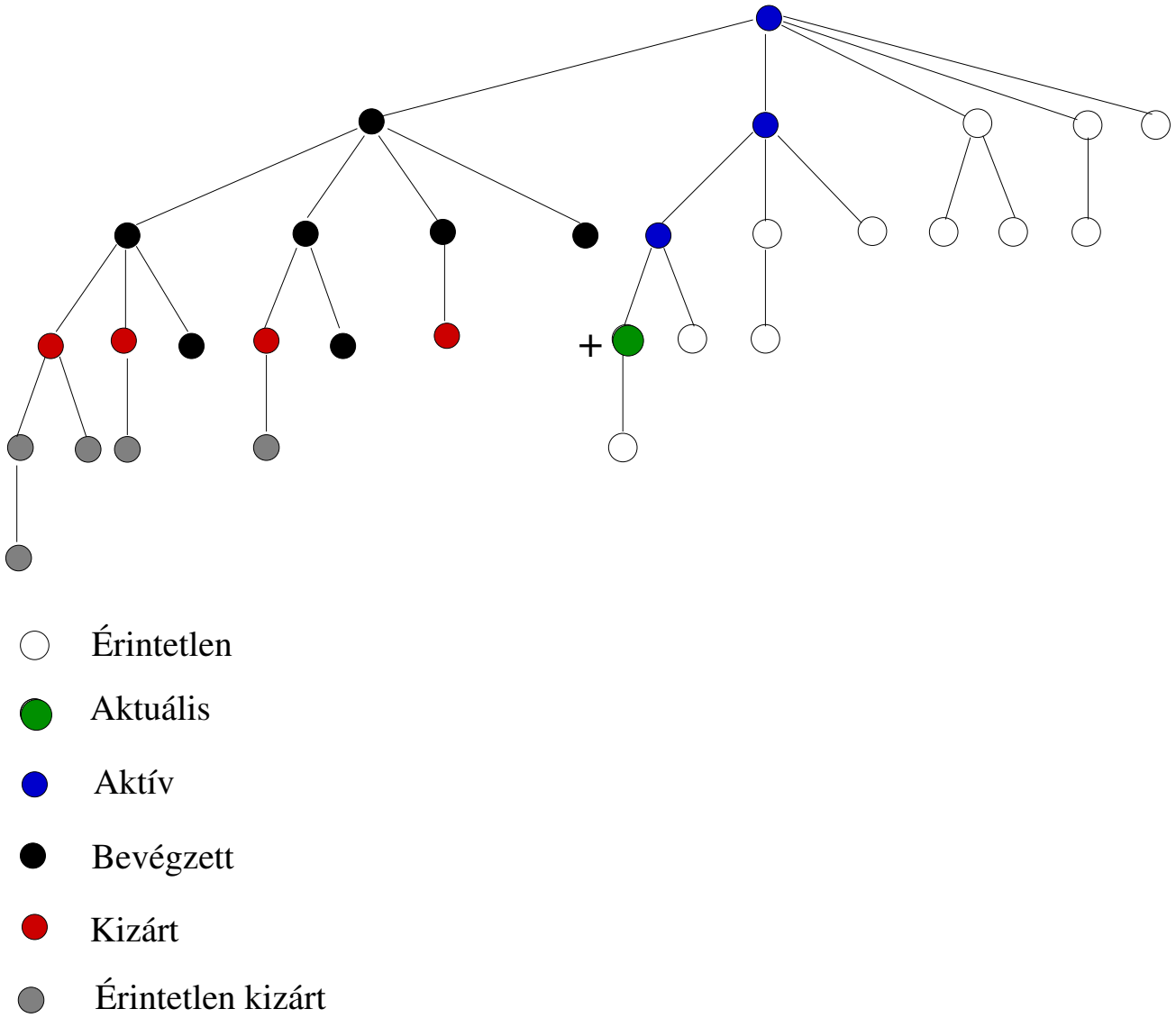
Rekurzív visszalépéses keresés algoritmusa

```
1  FaPont M=FaPont();//egy megoldás
2  bool van=false;
3  void rKeres(FaPont X){
4     if(van) return;//ha egy megoldást keresünk
5     if(!X.Lehet()) return;
6     if(X.Megoldas()){
7       M=X; X.Kilr();
8       van=true;
9       return;//ha egy megoldást keresünk
10    }//megoldást találtunk
11
12    if(!X.Elso()) return;
13    do{
14       rKeres(X);
15    }while(X.Testver());
16 }
```

Iteratív visszalépéses keresés algoritmusa

```
1  bool Keres(FaPont &X){
2     //Nemrekurzív visszalépéses keresés
3     bool tovabb=X.Lehet();
4     bool elsore=true;
5     while(tovabb){
6        if(elsore){
7           if(X.Megoldas()){return true;}//egy megoldás
8           elsore=X.Lehet() && X.Elso();//kizárás ha !Le
9        }else{
10          if(X.Testver())
11             elsore=true;
12          else//visszalépés
13             tovabb=X.Os();
14       }
15    }
16    return false;
17 }
```

A megoldástér pontjainak osztályozása



5. ábra. A megoldástér pontjainak osztályozása visszalépéses keresésnél

Érintetlen: az olyan pontja a megoldástérnek, amelyet a keresés során még nem érintettünk.

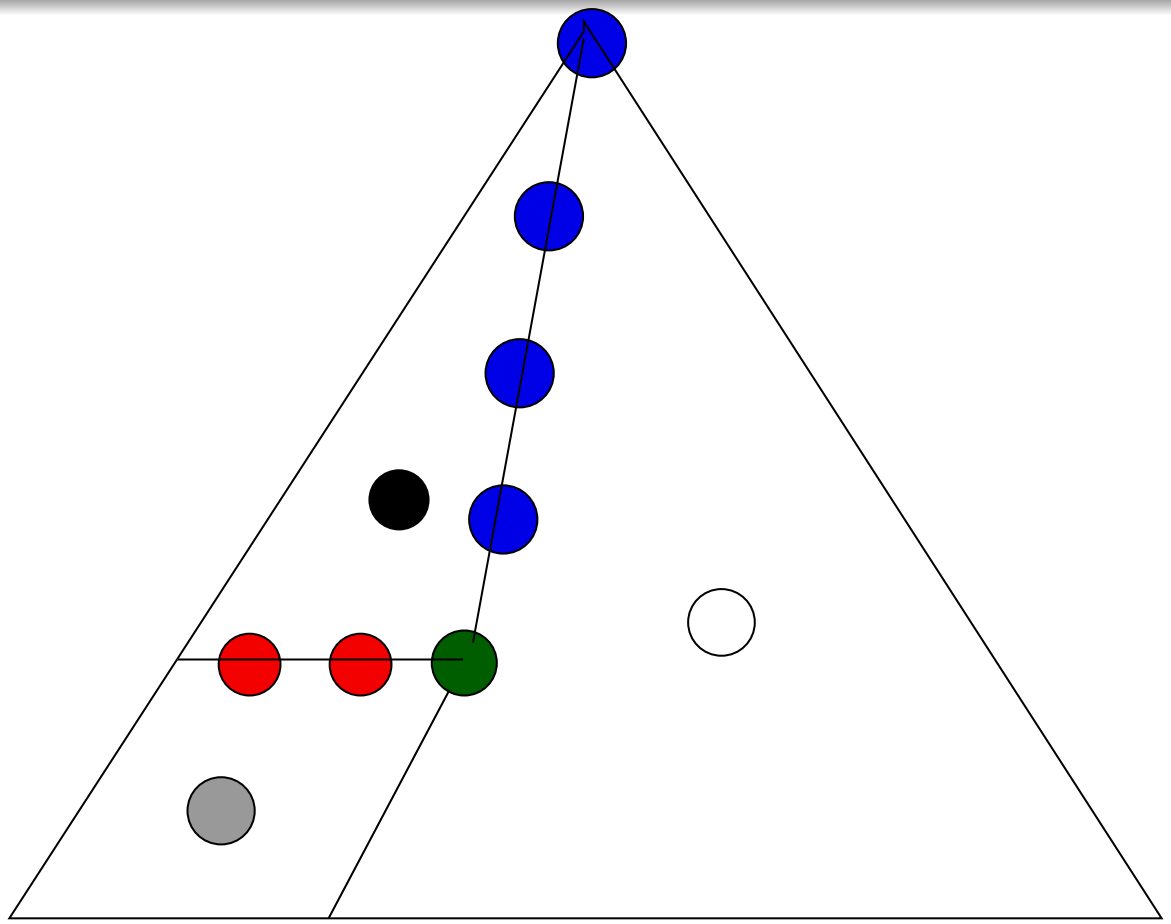
Aktuális: az a pont, amelyet éppen vizsgálunk.

Aktív az olyan pont, amelyet már érintettünk a keresés során, de még nem bevégzett. Pontosan azok az aktív pontok, amelyek az aktuális pont ősei a fában. A keresés során az aktív pontokba még visszatérünk.

Kizárt: olyan megoldáskezdemény, amelynek egyetlen folytatása (leszármazottja a fában) sem lehet megoldás.

Bevégzett: minden fia vagy kizárt, vagy bevégzett.

Érintetlen-kizárt: leszármazottja valamely kizárt pontnak. Tehát a megoldástér ezen pontjait nem érinti a keresés.

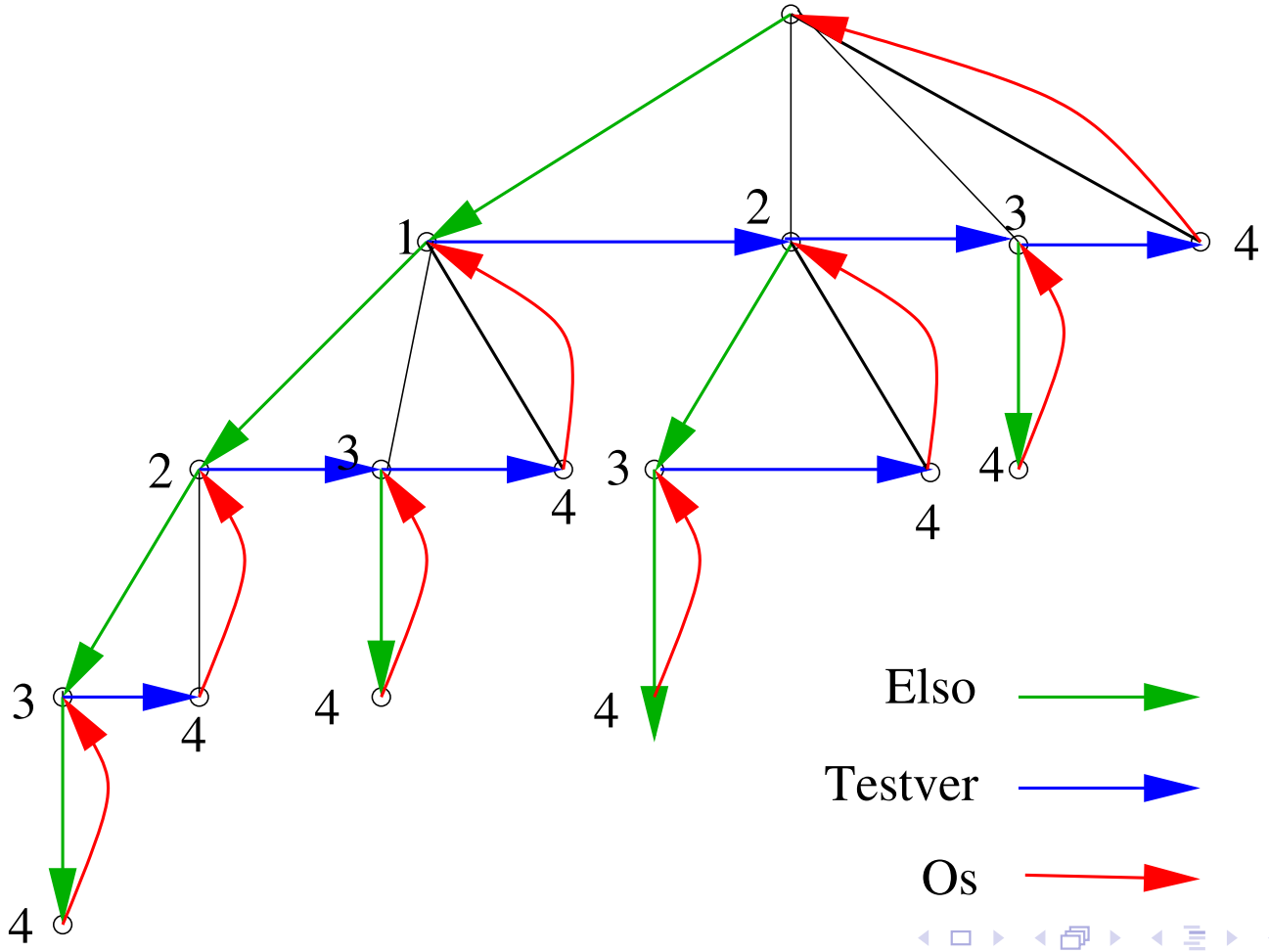


6. ábra. A megoldástér sematikus képe visszalépéses keresésnél.

A továbbiakban először megmutatjuk, hogyan lehet megvalósítani az egyes kombinatorikus struktúrák (permutációk, részhalmazok,

Fabejáró műveletek `részhalmaz` megoldás esetén

Az 1, ..., n alaphalmaz részhalmazai reprezentálhatók az elemek növekvő felsorolásával, vagy bitvektorral.

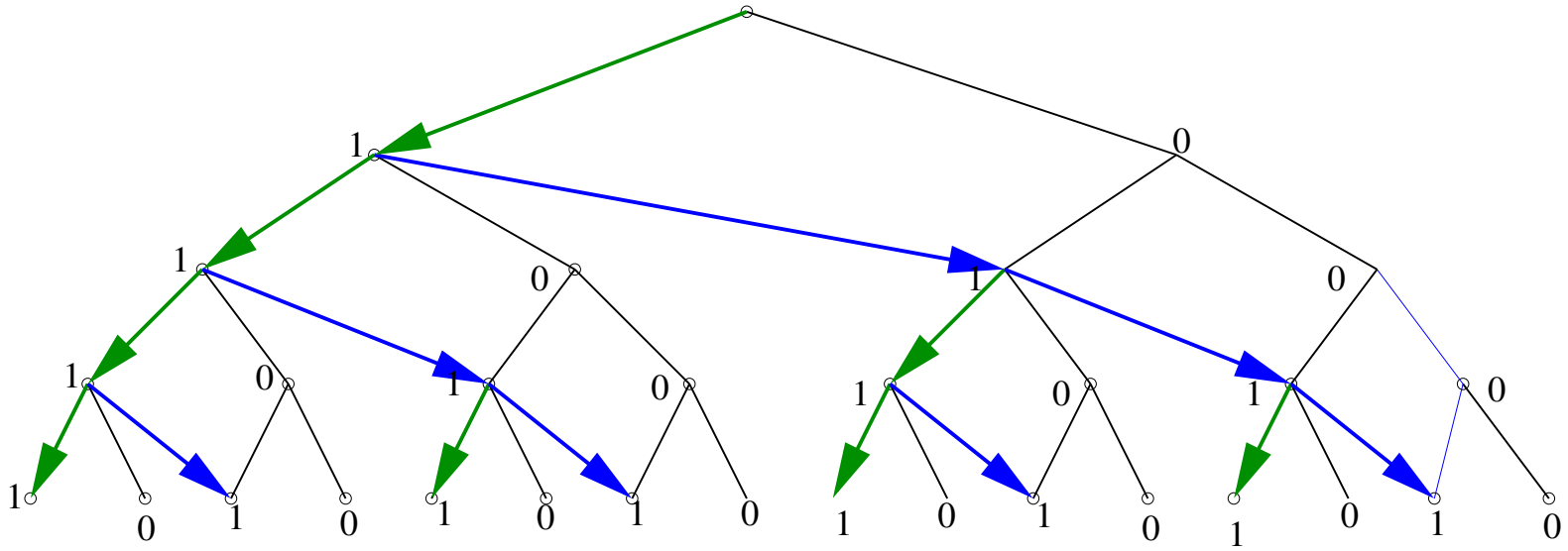


7. ábra. A megoldástér fája $n = 4$ esetén

```
1  class FaPont{//a megoldástér pontja
2    //Probléma-specifikus adattagok
3    int n;
4    vector<int> P;//egy részhalmaz
5    public:
6    bool Elso();
7    bool Testver();
8    bool Os();
9    FaPont(int xn){ n=xn; }
10   void Kilr(){
11     for(int x:P) cout<<x<<" "; cout<<endl;
12   }
13 };
14
15 bool FaPont::Elso(){
16   int u=P.size()==0 ? 0: P.back();
17   if(u==n) return false;
18   P.push_back(u+1);
19   return true;
20 }
```

```
bool FaPont::Testver(){
  if(P.size()==0 || P.back()==n) return false;
  P[P.size()-1]++;
  return true;
}

bool FaPont::Os(){
  if(P.size()==0) return false;
  P.pop_back();
  return true;
}
```



8. ábra. Részhalmazok bitvektoros reprezentációja $n = 5$ esetén

Minden részhalmazt olyan bitsorozat reprezentál, amelynek az utolsó tagja igaz értékű. Az Elso(R) és a Testver(R) műveletekkel generálhatjuk az összes részhalmazt.

```
1  class FaPont{//a megoldástér pontja
2    //Probléma-specifikus adatoktagok
3    int n;//az alaphalmaz: {1,...,n}
4    vector<int> P;//egy reszhalmaz
5    public:
6    bool Elso();
7    bool Testver();
8    bool Os();
9    FaPont(int xn){ n=xn;}
10   void Kilr(){
11     for(int i=0;i<P.size();i++)
12     if(P[i]) {cout<<i+1<<",";} cout<<endl;
13   }
14 };
15
16 bool FaPont::Elso(){
17   if(P.size()==n) return false;
18   P.push_back(1);
19   return true;
20 }
21
```

```
22  bool FaPont::Testver(){
23     if(P.size()==n) return false;
24     if(P.size()==0 || P[P.size()-1]==0) return false;
25     P[P.size()-1]=0;
26     P.push_back(1);
27     return true;
28  }
29
30  bool FaPont::Os(){
31     if(P.size()==0) return false;
32     P.pop_back();
33     return true;
34  }
```

```
12  void FaBejar(FaPont& F){
13     bool tovabb=true;
14     bool elsore=true;
15     while(tovabb){
16        if(elsore){
17           //a fapont feldolgozása
18           F.Kilr();
19           elsore=F.Elso();
20        }else{
21           if(F.Testver())
22           elsore=true;
23           else//nincs testvére, visszalépés
24           tovabb=F.Os();
25        }
26     }
27  }
```

```
28 void rFaBejar(FaPont X){
29    //rekurzív fabejárás
30    //az X fapont feldolgozása
31    X.Kilr();
32    if(!X.Elso()) return;
33    do{
34       rFaBejar(X);
35    }while(X.Testver());
36 }
```

A Pénzváltás feladat megoldása

Minden fapontban tároljuk kiegészítő adatként a megoldáskezdeményben lévő pénzek összegét a *resz* adattagban. Előzetesen számítsuk ki a $Maradt[i] = \sum_{j=i}^{n} Penz[j]$ szuffix összegeket. Ekkor korlátozó feltételként megadhatjuk azt, hogy $resz + Maradt[u+1] \geq K$, ahol u a megoldáskezdemény utolsó tagja (a legnagyobb pénz index).

```
1   //A Pénzváltás feladat megoldása
2   const int maxN=21;
3   int Penz[maxN];   //a pénzek
4   int Maradt[maxN];//szuffix összegek
5   int K;            //a kifizetendő
6   class FaPont{//a megoldástér pontja
7     //Globális: Penz[], K
8     //Probléma-specifikus adattagok
9     int n;  int resz=0;
10    vector<int> P;//egy részhalmaz
11    public:
12    bool Elso();
13    bool Testver();
14    bool Lehet();
15    bool Megoldas();
16    FaPont(int xn){ n=xn;}
17    void Kilr(){
18      for(int x:P) cout<<x<<" "; cout<<endl;}
19    FaPont(){};
20  };
```

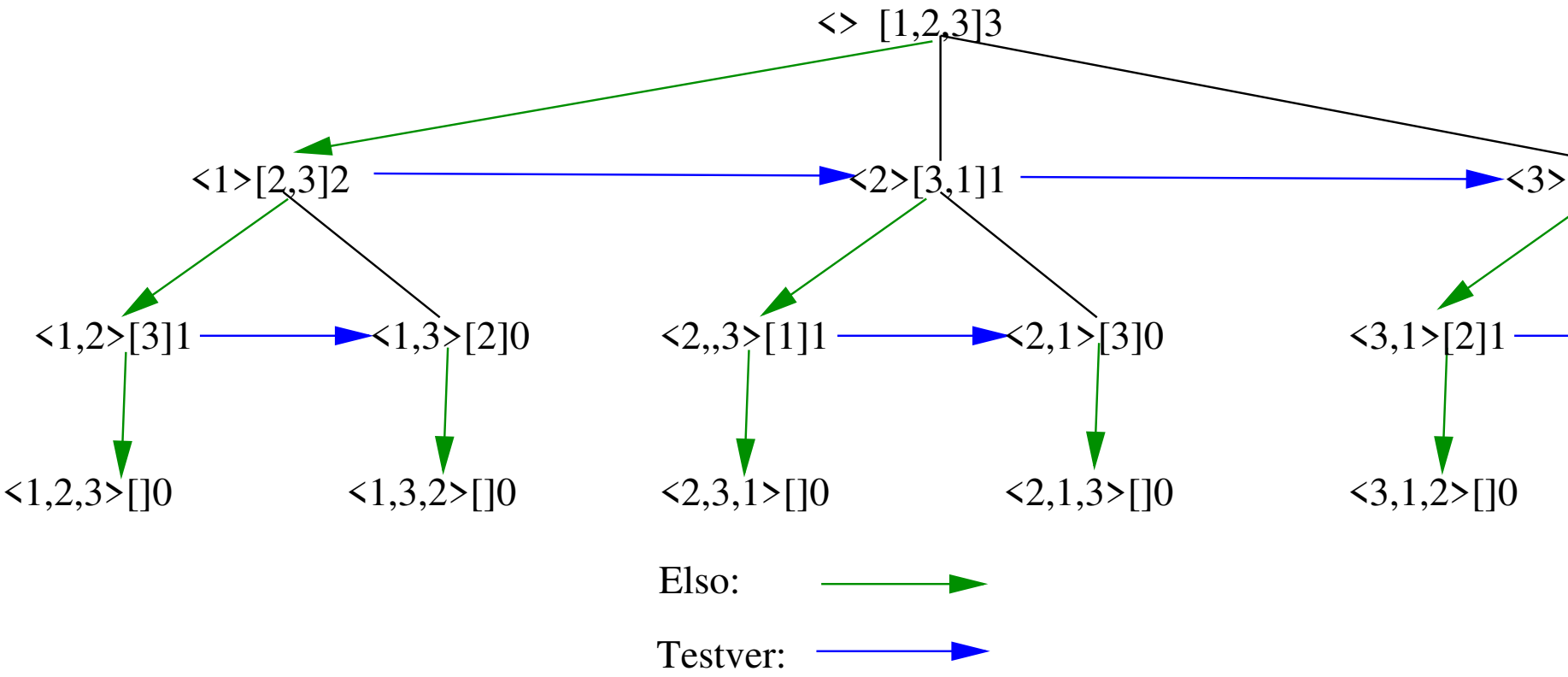
```
37    bool FaPont::Elso(){
38      int u=P.size()==0 ? 0: P.back();
39      if(u==n) return false;
40      P.push_back(u+1);
41      resz+=Penz[u+1];
42      return true;
43    }
44
45    bool FaPont::Testver(){
46      if(P.size()==0 || P.back()==n) return false;
47      resz-=Penz[P.back()];
48      P[P.size()-1]++;
49      resz+=Penz[P.back()];
50      return true;
51    }
52    bool FaPont::Megoldas(){return resz==K;}
53    bool FaPont::Lehet(){
54      int u=(P.size()==0) ? 1: P.back()+1;
55      if(u>n) return false;
56      return resz<=K && resz+Maradt[u]>=K;
57    }
```

```
58  FaPont M=FaPont();//egy megoldás
59  bool van=false;
60  void rKeres(FaPont X){
61     if(van) return;//ha egy megoldást keresünk
62     if(!X.Lehet()) return;
63     if(X.Megoldas()){
64       M=X;
65       van=true; //return;
66     }//megoldást találtunk
67
68     if(!X.Elso()) return;
69     do{
70       rKeres(X);
71     }while(X.Testver());
72  }
```

```
73    int main(){
74       int N;
75       cin>>K>>N;
76       for(int i=1;i<=N;i++){
77          cin>>Penz[i];
78          Maradt[i]=0;
79       }
80       Maradt[N]=Penz[N];
81       for(int i=N-1;i>=1;i--)
82       Maradt[i]=Maradt[i+1]+Penz[i];
83       FaPont F=FaPont(N);
84
85       rKeres(F);
86      M.Kilr();
87    }
```

Fabejáró műveletek permutáció megoldás esetén

Műveletek rekurzív bejáráshoz



9. ábra. A megoldástér fája $n = 3$ esetén

Négy adatot tárolunk a megoldástér pontjában: n a permutáció mérete, $P =<>$ a permutáció kezdőszelete, $Q = [\,]$ a P komplementere, tehát azon számok, amelyek még választhatók a permutáció következő tagjaként. A negyedik adattag az l, ami azt jelenti, hogy a Q sorozatból még az első l tag választható, amivel folytatható a P permutáció kezdőszelet. A Testver() művelettel mindig Q első tagját választjuk, P utolsó tagja pedig Q végére kerül. Ezért Q típusa kétvégű sor (deque) lesz. Ha $X =< p_1, \ldots, p_{k-1} > [p_k, \ldots, p_n]\, l$, akkor Elso(X) hatására $X =< p_1, \ldots, p_{k-1}, p_k > [p_{k+1}, \ldots, p_n]\, n-k$ lesz, a Testver(X) hatására pedig $X =< p_1, \ldots, p_{k-2}, p_k > [p_{k+1}, \ldots, p_n, p_{k-1}]\, l-1$.

```
1  class FaPont{//a megoldástér pontja
2    int n;//permutáció mérete
3    int l;//léptetések száma
4    vector<int> P;//egy permutáció kezdőszelete
5    deque<int> Q; //P komplementere
6    public:
7    bool Elso();
8    bool Testver();
9    FaPont(int xn){
10     n=xn;l=n;
11     for(int x=1;x<=n;x++) Q.push_back(x);
12   }
13   void Kilr(){for(int x:P) cout<<x<<" "; cout<<endl
14   }
15 };
```

```
88  bool FaPont::Elso(){
89     if(Q.size()==0) return false;
90     int q=Q.front(); Q.pop_front();
91     P.push_back(q); l=Q.size();
92     return true;
93  }
94  bool FaPont::Testver(){
95     if(l==0) return false;
96     int q=Q.front(); Q.pop_front();
97     int p=P.back(); P.pop_back();
98     P.push_back(q);
99     Q.push_back(p);
100    l--;
101    return true;
102 }
```

Műveletek iteratív bejáráshoz

Adott n-re az összes permutáció és azoknak a kezdőszeleteinek előállítása, azaz a megoldástér iteratív bejárásához szükséges Os() művelet nem oldható meg (hatékonyan=konstans időben) azokkal az adatokkal, amelyeket a rekurzív esetben alkalmaztunk. Ugyanis, nem tudjuk, hogy az ős pontban mi volt az l értéke, pedig azt vissza kell állítani. Tehát az l skalár érték helyett egy L veremben tároljuk az adott faponthoz tartozó l értéket. Az ős pont P és Q értéke visszaállítható, úgy, hogy P utolsó tagját átrakjuk Q végére, mert a Testver() művelettel cirkuláris léptetést alkalmaztunk.

```
1  class FaPont{//a megoldástér pontja
2    //Probléma-specifikus adattagok
3    int n;//permutáció mérete
4    vector<int> P;//megoldáskezdemény
5    deque<int> Q; //P komplementere
6    stack<int> L; //léptetések szintenként
7    public:
8    bool Elso();
9    bool Testver();
10   bool Os();
11   FaPont(int xn){
12     n=xn;
13     for(int x=1;x<=n;x++) Q.push_back(x);
14   }
15   void Kilr(){
16     for(int x:P) cout<<x<<" "; cout<<endl;
17   }
18 };
```

```
103 bool FaPont::Elso(){
104     if(Q.size()==0) return false;
105     int q=Q.front(); Q.pop_front();
106     P.push_back(q); L.push(Q.size());
107     return true;
108 }
109 bool FaPont::Testver(){
110     int l=L.top(); L.pop();
111     if(l==0){return false;}//nincs már testvér
112     int q=Q.front(); Q.pop_front();
113     int p=P.back(); P.pop_back();
114     P.push_back(q); Q.push_back(p);
115     l--; L.push(l);
116     return true;
117 }
118 bool FaPont::Os(){
119     if(P.size()==0) return false;
120     int p=P.back(); P.pop_back();
121     Q.push_back(p);
122     return true;
123 }
```

Az Ültetés feladat megoldása

Jelölje F feltétel párokat, tehát az i-edik tag azt kéri, hogy mellette vagy $F[i].first$, vagy $F[i].second$ üljön valamelyik oldalán. A szomszédság cirkulárisan értendő. Tehát az $1, \ldots, n$ halmaznak olyan permutációját keressük, amely minden tag kérését teljesíti, ha van ilyen.

Korlátozó feltételként megadhatjuk azt, hogy a megoldáskezdemény utolsó (nem n értékű) tagjának nem felel meg az előtte ülő, akkor következő tagként csak olyan választható, amely egy alkalmas szomszédja lehet az utolsónak.

```
1   //Az Ültetés feladat megoldása
2   #include <bits/stdc++.h>
3   using namespace std;
4   vector<pair<int,int>> F;
5
6   class FaPont{//a megoldástér pontja
7     //Globális: F
8     //Probléma-specifikus adattagok
9     int n;//permutáció mérete
10    int l;//léptetések száma
11    vector<int> P;//megoldáskezdemény
12    deque<int> Q; //P komplementere
13    public:
14    bool Elso();
15    bool Testver();
16    bool Lehet();
17    bool Megoldas();
18    FaPont(int xn){
19      n=xn;l=n;
20      for(int x=1;x<=n;x++) Q.push_back(x);
21    }
```

```
22        FaPont(){};
23        void Kilr(){
24           for(int x:P) cout<<x<<" "; cout<<endl;
25        }
26     };
27
28     bool FaPont::Elso(){
29        if(Q.size()==0) return false;
30        int q=Q.front(); Q.pop_front();
31        P.push_back(q);
32        l=Q.size();
33        return true;
34     }
35     bool FaPont::Testver(){
36        if(l==0) return false;
37        int q=Q.front(); Q.pop_front();
38        int p=P.back(); P.pop_back();
39        P.push_back(q); Q.push_back(p);
40        l--;
41        return true;
42     }
```

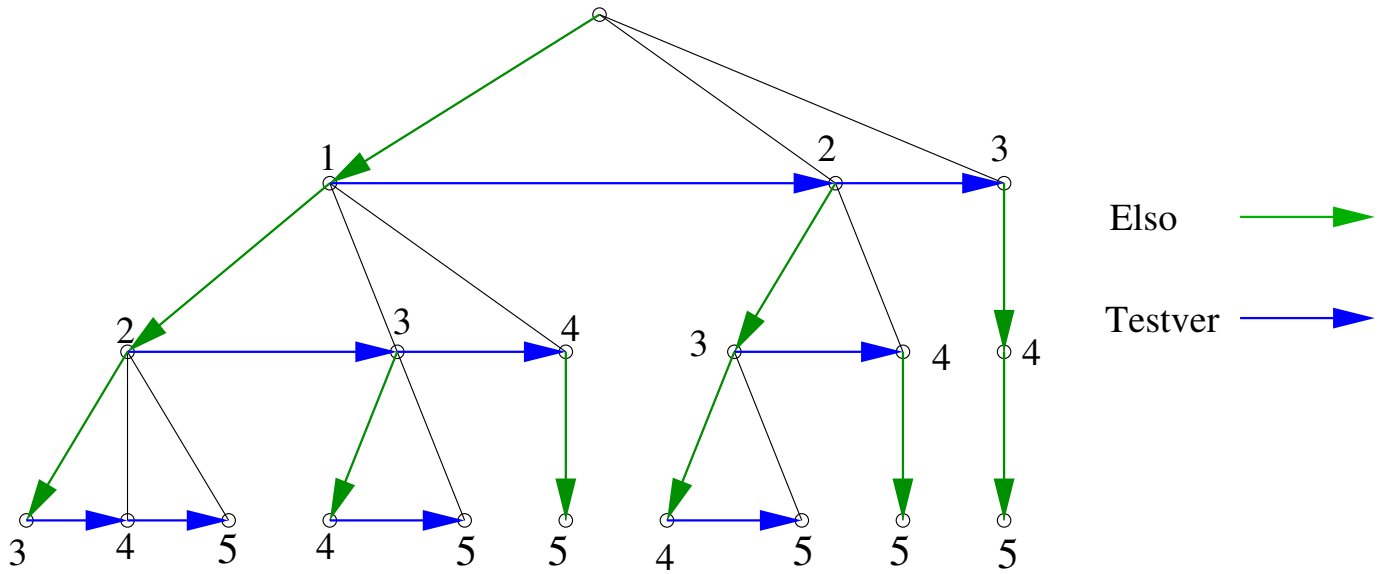
```
43    bool FaPont::Lehet(){
44      if(P.size()==n) return false;
45      if(P.size()<=1) return true;
46      int u=P.back(); int u1=P[P.size()-2];
47      int e=Q.front();
48      return  F[u].first==u1 || F[u].second==u1 ||
49          F[u].first==e || F[u].second==e ;
50    }
51  bool FaPont::Megoldas(){
52    if(P.size()<n) return false;
53    int kovet,eloz; bool van=true;
54    for(int i=0;i<n;i++){
55      kovet=P[(i+1)%n]; eloz= i>0 ? P[i-1]: P[n-1];
56      if( F[P[i]].first!=kovet &&
57        F[P[i]].second!=kovet &&
58        F[P[i]].first!=eloz && F[P[i]].second!=eloz){
59        van=false; break;
60      }
61    }
62    return van;
63  }
```

```
124     FaPont M=FaPont();//egy megoldás
125     bool van=false;
126     void rKeres(FaPont X){
127       if(van) return;//ha egy megoldást keresünk
128       if(!X.Lehet()) return;
129       if(X.Megoldas()){
130         M=X; X.Kilr();
131         van=true; //return;
132       }//megoldást találtunk
133
134       if(!X.Elso()) return;
135       do{
136         rKeres(X);
137       }while(X.Testver());
138     }
```

```
139    int main(){
140       int n,x,y;
141       F.push_back({0,0});
142       cin>>n;
143       for(int i=1;i<=n;i++){
144          cin>>x>>y;
145          F.push_back({x,y});
146       }
147       FaPont F=FaPont(n);
148       rKeres(F);
149       //cout<<"mego\n";
150       if(van) M.Kilr();
151    }
```

Fabejáró műveletek `kombináció` megoldás esetén

Az (n, k) kombinációk pontosan k elemű részhalmazok, ezért az összes kombináció, illetve a kezdőszeleteik előállítása hasonló a részhalmazokéhoz.



10. ábra. $(5, 3)$ kombinációk fája.

Műveletek rekurzív bejáráshoz

```
1  class FaPont{//a megoldástér pontja
2    //Probléma-specifikus adatoktagok
3    int n;//
4    int k;//(n,k) kombinációk
5    vector<int> P;//egy reszhalmaz
6    public:
7    bool Elso();
8    bool Testver();
9    FaPont(int xn, int xk){
10     n=xn; k=xk;
11   }
12   void Kilr(){
13     for(int x:P) cout<<x<<" "; cout<<endl;
14   }
15 };
```

```
152 bool FaPont::Elso(){
153     int l=P.size();
154     if(l==k) return false;
155     int p=(l==0) ? 0 : P.back();
156     if(p+k-l>n) return false;
157     P.push_back(p+1);
158     return true;
159 }
160 bool FaPont::Testver(){
161     int l=P.size();
162     if(l==0) return false;
163     int p= P.back();
164     if(p+k-l>=n) return false;
165     P[l-1]++;
166     return true;
167 }
```

Műveletek iteratív bejáráshoz

Az Elso() és a Testver() művelet ugyanaz, mint a rekurzív esetben,

```
1 bool FaPont::Os(){
2    int l=P.size();
3    if(l==0) return false;
4    P.pop_back();
5    return true;
6 }
```

A Hozzárendelés feladat megoldása

Jelölje $VM[v]$ azon munkák halmazát, amelyeket a v alvállalkozó el tud végezni.
A feladat megoldása megadható egy olyan $f : \{1, \ldots, M\} \to \{1, \ldots, V\}$ függvénnyel, amelyre teljesül az alábbi két feltétel:

$$f(i) \neq f(j) \text{ minden } 1 \leq i < j \leq M \text{ esetén}$$

$$m \in VM[f(m)] \text{ minden } 1 \leq m \leq M \text{ esetén}$$

Tehát a megoldás megadható az $(f(1), \ldots, f(M))$ sorozattal, ami nem más, mint V-nek M-ad osztályú variációja. Tehát minden lehetséges megoldása a feladatnak megadható egy-egy variációval. Mivel V értéke nem nagy. ezért minden $VM[v]$ halmazt reprezentálhatunk bitvektorral.

```
1  const int maxN=21;
2  bool VM[maxN][maxN];
3  //a v tudja az m munkát, ha V[v][m] igaz
4  class FaPont{//a megoldástér pontja
5    //Globális: VM[][]
6    //Probléma-specifikus adatoktagok
7    int n;//
8    int k;//(n,k) variácok fája
9    int l;//testver léptetések száma
10   vector<int> P;//egy reszhalmaz
11   deque<int> Q; //P komplementere
12   public:
13   bool Elso();
14   bool Testver();
15   bool Lehet();
16   bool Megoldas();
17   FaPont(int xn, int xk){
18     n=xn; k=xk; l=n;
19     for(int x=1;x<=n;x++) Q.push_back(x);
20   }
```

```
169     FaPont(){};
170     void Kilr(){
171       for(int x:P) cout<<x<<" "; cout<<endl;}
172   };
173   bool FaPont::Elso(){
174     if(P.size()==k) return false;
175     if(Q.size()==0) return false;
176     int q=Q.front(); Q.pop_front();
177     P.push_back(q);
178     l=Q.size();
179     return true;
180   }
181   bool FaPont::Testver(){
182     if(l==0) return false;
183     int q=Q.front(); Q.pop_front();
184     int p=P.back(); P.pop_back();
185     P.push_back(q);
186     Q.push_back(p);
187     l--;
188     return true;
189   }
```

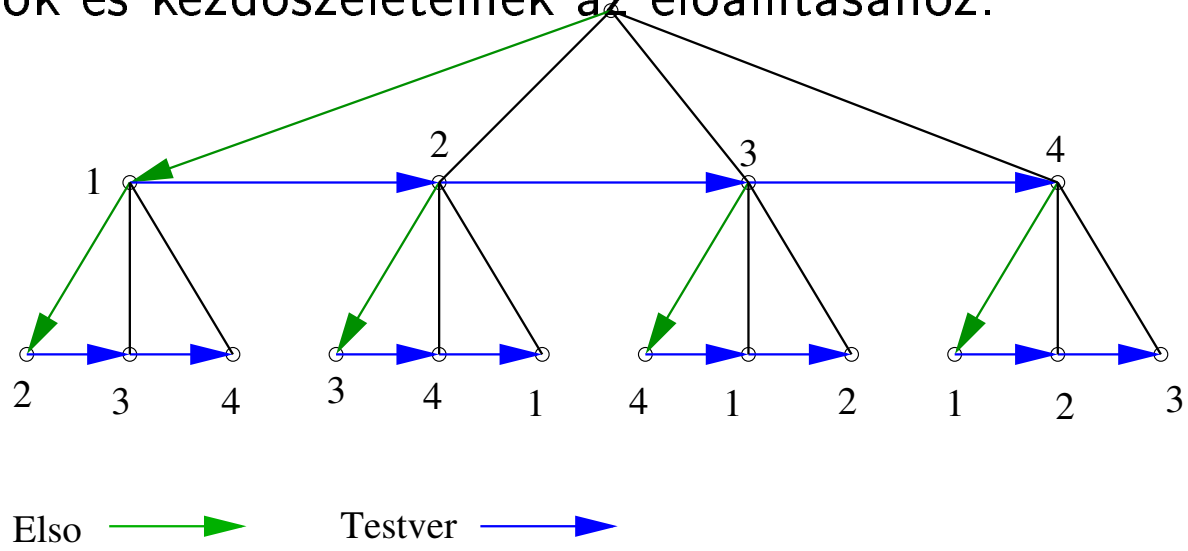
```
190
191 bool FaPont::Lehet(){
192   //   return true;
193   if(P.size()==0) return true;
194   return VM[P.size()][P.back()];
195 }
196
197 bool FaPont::Megoldas(){
198   return (P.size()==k);
199 }
```

```
200  FaPont M=FaPont();//egy megoldás
201  bool van=false;
202  void rKeres(FaPont X){
203    if(van) return;//ha egy megoldást keresünk
204    if(!X.Lehet()) return;
205
206    if(X.Megoldas()){
207      M=X; X.Kilr();
208      van=true; return;
209    }//megoldást találtunk
210    if(!X.Elso()) return;
211    do{
212      rKeres(X);
213    }while(X.Testver());
214  }
```

```
215 int main(){
216   int m,v,x;
217   string s;
218   cin>>m>>v;
219   getline(cin, s);
220   for(int i=1;i<=v;i++)
221   for(int j=1;j<=m; j++) VM[j][i]=false;
222   for(int i=1;i<=v;i++){
223     while(cin.peek()!='\n'){
224       cin>>x;
225       VM[x][i]=true;
226     }
227     getline(cin, s);
228   }
229   FaPont VM=FaPont(v, m);
230   rKeres(VM);
231   if(van) M.Kilr();
232 }
```

Fabejáró műveletek `variáció` megoldás esetén

Mivel a (n, k) variációk az n méretű permutációk k méretű különböző kezdőszeletei, ezért az előállításuk hasonló a permutációk és kezdőszeleteinek az előállításához.



11. ábra. $(4, 2)$ vatiációk fája

Műveletek rekurzív bejáráshoz

```
1  class FaPont{//a megoldástér pontja
2    //Probléma-specifikus adatoktagok
3    int n;//
4    int k;//(n,k) variácok fája
5    int l;//testver léptetések száma
6    vector<int> P;//egy reszhalmaz
7    deque<int> Q; //P komplementere
8    public:
9    bool Elso();
10   bool Testver();
11   FaPont(int xn, int xk){
12     n=xn; k=xk; l=n;
13     for(int x=1;x<=n;x++) Q.push_back(x);
14   }
15   void Kilr(){
16     for(int x:P) cout<<x<<" "; cout<<endl;
17   }
18 };
19
```

```
20  bool FaPont::Elso(){
21     if(P.size()==k) return false;
22     if(Q.size()==0) return false;
23     int q=Q.front(); Q.pop_front();
24     if(q==0) return false;
25     P.push_back(q);
26     l=Q.size();
27     return true;
28  }
29  bool FaPont::Testver(){
30     if(l==0) return false;
31     int q=Q.front(); Q.pop_front();
32     int p=P.back(); P.pop_back();
33     P.push_back(q);
34     Q.push_back(p);
35     l--;
36     return true;
37  }
```

Műveletek iteratív bejáráshoz

```
1  class FaPont{//a megoldástér pontja
2    //Probléma-specifikus adatoktagok
3    int n;//
4    int k;//(n,k) variácok fája
5    vector<int> P;//egy reszhalmaz
6    deque<int> Q; //P komplementere
7    stack<int> L; //léptetesek szintenként
8    public:
9    bool Elso();
10   bool Testver();
11   bool Os();
12   FaPont(int xn, int xk){
13     n=xn; k=xk;
14     for(int x=1;x<=n;x++) Q.push_back(x);
15   }
16   void Kilr(){
17     for(int x:P) cout<<x<<" "; cout<<endl;
18   }
19 };
```

```
20
21 bool FaPont::Elso(){
22    if(Q.size()==0) return false;
23    if(P.size()==k) return false;
24    int q=Q.front(); Q.pop_front();
25    P.push_back(q);
26    L.push(Q.size());
27    return true;
28 }
29 bool FaPont::Testver(){
30    int l=L.top(); L.pop();
31    if(l==0){return false;}
32    int q=Q.front(); Q.pop_front();
33    int p=P.back(); P.pop_back();
34    P.push_back(q);
35    Q.push_back(p);
36    l--; L.push(l);
37    return true;
38 }
39 bool FaPont::Os(){
40    if(P.size()==0) return false;
```

```
41     int p=P.back(); P.pop_back();
42    Q.push_back(p);
43     return true;
44  }
```

A Koalíciók feladat megoldása

Minden k tagú koalíció egy (p, k) kombinációval adható meg, ahol p a parlamenti pártok száma. Az összes (p, k) kombinációt sorra kell venni, és ha többségi, akkor növeljük minden tagjának a megoldás számlálóját.

```
1  vector<int> M;//mandátumok szána
2  int Ossz=0;
3  vector<int> K;
4  //a p párt K[p] koalícióban lehet tag
5  class FaPont{//a megoldástér pontja
6    //Globális: Ossz, K
7    //Probléma-specifikus adatoktagok
8    int n,k;
9    public:
10   vector<int> P;//egy reszhalmaz
11   bool Elso();
12   bool Testver();
13   bool Lehet();
14   bool Megoldas();
15
16   FaPont(int xn, int xk){n=xn; k=xk;}
17   void Kilr(){
18     for(int x:P) cout<<x<<" "; cout<<endl;
19   }
20   FaPont(){};
21 };
```

```
22
23 bool FaPont::Elso(){
24    int l=P.size();
25    if(l==k) return false;
26    int p=(l==0) ? 0 : P.back();
27    if(p+k-l>n) return false;
28    P.push_back(p+1);
29    return true;
30 }
31 bool FaPont::Testver(){
32    int l=P.size();
33    if(l==0) return false;
34    int p= P.back();
35    if(p+k-l>=n) return false;
36    P[l-1]++;
37    return true;
38 }
39 bool FaPont::Lehet(){
40    return P.size()<=k;
41 }
42
```

```
43  bool FaPont::Megoldas(){
44     if(P.size()!=k) return false;
45     int om=0;
46     for(int x:P) om+=M[x];
47     return 2*om>Ossz;
48  }
49
50  void rKeres(FaPont X){
51     if(!X.Lehet()) return;
52     if(X.Megoldas()){
53        for(int x:X.P) K[x]++;
54     }//megoldást találtunk
55
56     if(!X.Elso()) return;
57     do{
58        rKeres(X);
59     }while(X.Testver());
60  }
```

```
234 int main(){
235    int p,k,x;
236    cin>>p>>k;
237    K.push_back(0);
238    M.push_back(0);
239    for(int i=1;i<=p;i++){
240       cin>>x;
241       M.push_back(x);
242       K.push_back(0);
243       Ossz+=x;
244    }
245    FaPont F=FaPont(p, k);
246    rKeres(F);
247    for(int i=1;i<=p;i++) cout<<K[i]<<" "; cout<<endl
248 }
```

Visszalépéses keresési algoritmusok futási ideje

Legrosszabb esetben a keresés a megoldástér-fa minden pontját bejárja, aminek mérete általában exponenciális a megoldásvektor méretének függvényében.

Visszalépéses keresési algoritmusok tárigénye

Rekurzív megoldás esetén a rekurzió mélysége határozza meg a felhasznált memóriát.
Iteratív megvalósítás esetén a megoldástérnek csak egyetlen pontját kell tárolni.
A visszalépéses keresés alkalmazható optimális megoldás előállítására is.
Legyen $Cf(X)$ a célfüggvény, tehát olyan X-et keresünk, amelyre: $Megoldas(X)$ és $Cf(X)$ minimális.
Az összes megoldás keresése során ha jobb megoldást találunk, feljegyezzük.

Megoldás keresés elágazás-korlátozással (Branch and bound)

Úgy tekintjük, hogy a megoldásteret alkotó megoldáskezdemények fát alkotnak, ezért minden fabejáró módszer (algoritmus) alkalmazható megoldás keresésére. A visszalépéses keresés a mélységi bejárás stratégiáját alkalmazza, aminek az a lényege, hogy a keresés során mindig egy aktív pontot egy lehetséges folytatásával folytatja a keresés, és visszatérést hajt végre, ha nem lehet folytatni. Nyilvánvalóan lehet alkalmazni olyan bejárási stratégiát, amely az aktív pont minden lehetséges közvetlen folytatását kigenerálja, ezeket tárolja alkalmas tárolóban és innen választ következő aktív pontot. Ezen stratégia algoritmusa megfogalmazható általánosan, ahol az aktív pontokat egy adagoló tárolja.
Adagolón olyan adattípust értünk, amelynek értékhalmaza adott E elemtípusbeli véges halmazok és az alábbi három művelete van:

void Betesz(E X)
Az X adatelemet beteszi a halmazba.
E Kivesz() A halmaz egy tetszőleges elemét adja vissza, amelyet kivesz a halmazból.
bool Ures()
Akkor és csak akkor ad igaz értéket, ha a halmaz üres.

Megoldás keresése adagolós bejárással

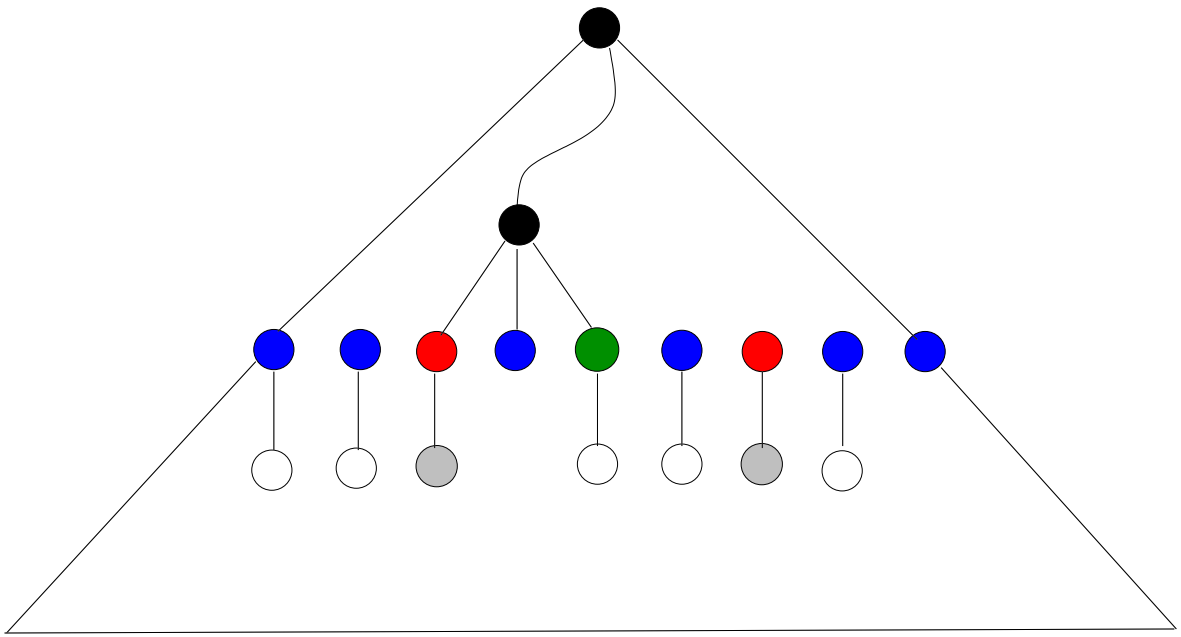
```
1 class FaPont{
2   //probléma-specifikus adatoktagok
3   public:
4   bool Elso(){};
5   bool Testver(){};
6   bool Lehet(){};
7   bool Megoldas(){};
8   FaPont(){};
9 };
```

```
249 class Adagolo{
250   public:
251   void Betesz(FaPont X){};
252   FaPont Kivesz(){};
253   bool Ures(){};
254 };
```

```
255 FaPont AKeres(FaPont X){
256   //Egy megoldás keresése az X gyökerű
257   //megoldástér-fában
258   Adagolo A;
259   FaPont M;//egy megoldás
260   if (!X.Lehet()) return M;
261   A.Betesz(X);
262   while (!A.Ures()){
263     X=A.Kivesz();
264     //X közvetlen leszármazottainak (gyerekeinek) a
265     if(!X.Elso()) continue;//X-nek nincs folytatása
266     do{//X minden testvérére
267       if (!X.Lehet()) continue;//kizárás
268       if (X.Megoldas()){//megoldást találtunk
269         M=X; break;//ha egy megoldást kerestünk
270       }
271       A.Betesz(X);
272     }while(X.Testver());
273   }//while
274   return M;
275 }
```

Az adagoló lehet (többek között) verem, sor vagy prioritási sor. Sőt, az aktuális pontot akár véletlenszerűen is választhatjuk az aktívak közül. Tágabb értelemben minden olyan megoldási módszert, amely adagolóval történő keresést alkalmaz, elágazás-korlátozás elnevezéssel illetnek (Brach and Bound). Szűkebb értelemben csak azokat a módszereket, amelyeknél az adagoló prioritási sor. A módszert leginkább optimalizációs feladatok megoldására alkalmazzák, amikor bemeneti adatként adott egy Cf célfüggvény, és olyan megoldást keresünk, amelyre Cf értéke minimális/maximális.

A megoldástér pontjainak osztályozása



- Érintetlen
- Aktuális
- Aktív
- Kizárt
- Bevégzett
- Érintetlen-kizárt

12. ábra. Megoldástér elágazás-korlátozás bejárás során.

Ugyanaz a korlátozás módszere alkalmazható, mint amit a visszalépéses keresés során alkalmaztunk a Lehet() művelettel.
Ha prioritási sort használunk adagolónak és célfüggvény szerinti minimális megoldást keresünk, akkor további korlátozásokat alkalmazhatunk.
Tegyük fel, hogy minimalizációs feladatot kell megoldani. Tehát adott a $Cf(X)$ valós értékű célfüggvény, és olyan X megoldást keresünk, amelyre a célfüggvény $Cf(X)$ értéke minimális.
Tegyük fel továbbá, hogy a megoldáskezdeményekre meg tudunk adni olyan $AK(X)$ *alsó korlát* függvényt, amelyekre teljesülnek az alábbi egyenlőtlenségek.
Bármely X megoldáskezdeményre és minden olyan Y megoldásra, amely leszármazottja X-nek:

$$AK(X) \leq Cf(Y)$$

Ekkor az adagoló lehet az AK szerinti minimumos prioritási sor, tehát az aktív pontok közül mindig a legkisebb alsó korlátú pontot választjuk aktuálisnak.

Az keresés során tároljuk az addig megtalált megoldások célfüggvény értékeinek minimumát.

$$G_F_K = \mathtt{min}\{\, Cf\,(X)\ :\ Megoldas\,(X)\ \text{ és } X \text{ bevégzett}\,\}$$

Ekkor a következő kizárásokkal (korlátozásokkal) gyorsíthatjuk a keresést:

1. Ha az X új aktuális pontra $G_F_K < AK\,(X)$, akkor X kizárt lehet.
2. Ha a sorból az X pontot vettük ki és $G_F_K < AK\,(X)$, akkor a keresést befejezhetjük, hiszen nem kaphatunk már jobb megoldást.

Megoldás keresése alsó korlát szerinti minimumos prioritási sorral

```
1  FaPont BBKeres1(FaPont X){
2     int G_F_K=INT_MAX;
3     priority_queue<FaPont, vector<FaPont>,Rendezo> S;
4     FaPont M;
5     if (!X.Lehet()) return M;
6     S.push(X);
7     while (S.size()>0 ){
8        X=FaPont(S.top()); S.pop();
9        if (G_F_K<=X.AK()) return M;
10       if(!X.Efiu()) continue;
11       do{
12          if (X.Lehet() && X.AK()<G_F_K ){
13             if (X.Megoldas() && X.Cf()<G_F_K){//X jobb
14                G_F_K=X.Cf(); M=X; }
15             S.push(X);
16          }//else X kizárt lesz
17       }while(X.Testver());
18    }//while
19    return M;
20 }
```

Megoldás keresése felső korlát szerinti minimumos prioritási sorral

Ha a megoldáskezdeményekre meg tudunk adni felső korlát is, akkor az adagoló lehet a felső korlát szerinti minimumos prioritási sor. Felső korlát olyan $FK(X)$ függvényt értünk, amelyre teljesül, hogy minden X megoldáskezdeményre és minden olyan Y megoldásra, amely leszármazottja X-nek:

$$Cf(Y) \leq FK(X)$$

Ekkor azonban a 2. kizárást nem alkalmazhatjuk.
Az $FK(X)$ felső korlátot erős felső korlátnak nevezzük, ha bármely X megoldáskezdeményre:

$$FK(X) < \infty \Rightarrow (\exists Y)(Y \trianglelefteq X \wedge Megoldas(Y) \wedge Cf(Y) \leq FK(X))$$

Az erős felső korlát létezése azt jelenti, hogy bármely X megoldáskezdeményre, ha $FK(X) < \infty$, akkor biztosan létezik megoldás az X gyökerű megoldástér fában, és ennek célfüggvény értéke $\leq FK(X)$. Ekkor a keresést végezhetjük a felső korlát

szerinti minimumos prioritási sorral úgy, hogy feljegyezzük az addig érintett pontok felső korlátjainak minimumát. Legyen ez a G_F_K. Tehát a keresés során azt állíthatjuk, hogy biztosan létezik olyan megoldás, amelynek célfüggvény értéke $\leq G_F_K$, de nem biztos, hogy már találtunk is ilyen megoldást. A 2. számú kizárást ennek ellenére továbbra is megtehetjük.

```
1   FaPont BBKeres2(FaPont X){
2     int G_F_K=INT_MAX;
3     priority_queue<FaPont, vector<FaPont>,Rendezo>
4     FaPont M;//az optimális megoldás
5     if (!X.Lehet()) return M;
6     S.push(X);
7     while (S.size()>0 ){
8       X=FaPont(S.top()); S.pop();
9       if (G_F_K<X.AK()) continue;//X kizárt lett
10      if(!X.Efiu()) continue; //X kizárt lett
11      do{
12        if (X.Lehet() && X.AK()<G_F_K ){
13          if (X.Megoldas() && X.Cf()<=G_F_K){
14            G_F_K=X.Cf(); M=X;
15          }else if(X.FK()<G_F_K)  G_F_K=X.FK();
16          S.push(X);
17        }//else X kizárt lesz
18      }while(X.Testver());
19    }//while
20    return M;
21  }
```

Az elágazás-korlátozás módszer alkalmazása az optimális pénzváltás probléma megoldására.

Probléma: Optimális pénzváltás

Bemenet: $P = \{p_1, \ldots, p_n\}$ pozitív egészek halmaza, és K pozitív egész szám, a kifizetendő összeg.

Kimenet: Olyan $S \subseteq P$, hogy $\sum_{p \in S} = K$ és $|S| \rightarrow$ minimális.

Tegyük fel, hogy a pénzek nagyság szerint nemcsökkenő sorrendbe rendezettek: $p_1 \geq \ldots \geq p_n$. A megoldást keressük $X = \langle i_1, \ldots, i_m \rangle$, $i_1 < i_2 < \ldots < i_m$ alakú vektor formában. jelölje $Resz = \sum_{k=1}^{m} p_{i_k}$ és $Maradt = \sum_{j=i_m+1}^{n} p_j$ összegeket.

$$AK(X) = m + \lceil (K - Resz) / p_{i_m+1} \rceil$$

$$FK(X) = m + \lceil (K - Resz) / p_n \rceil$$

Ez a felső korlát nem erős, és nem is tudunk olcsón kiszámítható erős felső korlátot adni.

```
1  //Az optimális pénzváltás algoritmusa
2  #include<bits/stdc++.h>
3  using namespace std;
4  vector<int> P;//a pénzek
5  int K; //a kifizetendő érték
6
7  class FaPont{
8    //Globális: K, P
9    //probléma-specifikus adattagok
10   int resz=0; int maradt=0;
11   vector<int> A; //megoldáskezdemény
12   public:
13   bool Elso();
14   bool Testver();
15   bool Lehet();
16   bool Megoldas();
17   int AK();//az alsó korlát függvény
```

```
276     FaPont(){for(int x:P) maradt+=x;};
277     int Cf(){return A.size();}//a célfüggvény
278     void Kilr(){
279       if(A.size()==0)
280       cout<<"Nincs megoldás\n";
281       else{
282         cout<<K<<"="; for(int x:A) cout<<P[x]<<"+"; c
283       }
284     }
285   };
286   int FaPont:: AK(){
287     if(A.size()==0)
288     return (K+P[P.size()-1]-1)/P[P.size()-1];
289     if(A[A.size()-1]==P.size()-1){return A.size();};
290     return A.size()+(K-resz+P[A[A.size()-1]+1]-1)/P[A
291   }
292   struct Rendezo{//a prioritási sor rendezési reláció
293     Rendezo(){};
294     bool operator() (FaPont& X, FaPont& Y) const{
295       return X.AK()>Y.AK();
296     }};
```

```
297 FaPont BBKeres(FaPont X){
298   int G_F_K=INT_MAX;
299   priority_queue<FaPont, vector<FaPont>, Rendezo> S
300   FaPont M;
301   if (!X.Lehet()) return M;
302   S.push(X);
303   while (S.size()>0 ){
304     X=FaPont(S.top()); S.pop();
305     if (G_F_K<=X.AK()) return M;
306     if(!X.Elso()) continue;
307     do{
308       if (X.Lehet() && X.AK()<G_F_K ){
309         if (X.Megoldas() && X.Cf()<G_F_K){//X jobb
310           G_F_K=X.Cf(); M=X;
311         }
312         S.push(X);
313       }//else X kizárt lesz
314     }while(X.Testver());
315   }//while
316   return M;
317 }
```

```
318 bool FaPont::Lehet(){
319   return resz<=K && K<=resz+maradt;
320 }
321 bool FaPont::Elso(){
322   if(A.size()==0){
323     A.push_back(0);
324     resz=P[0];
325     maradt-=P[0];
326     return true;
327   }
328   int x=A[A.size()-1];
329   if(x<P.size()-1){
330     A.push_back(x+1);
331     resz+=P[x+1];
332     maradt-=P[x+1];
333     return true;
334   }else return false;
335 }
```

```
336 bool FaPont::Testver(){
337   if(A.size()==0) return false;
338   int x=A[A.size()-1];
339   if(x<P.size()-1){
340     resz-=P[x]; resz+=P[x+1];
341     maradt-=P[x+1];
342     A[A.size()-1]=x+1;
343     return true;
344   }else{
345     return false;
346   }
347 }
348 bool FaPont::Megoldas(){return K==resz;}
349
350 int main(){
351   P={9,6,3,2,1}; //a pénzek
352   K=12; //a kifizetendő érték
353   FaPont X=FaPont();//X a megoldástér gyökere
354   X=BBKeres(X);
355   X.Kilr();
356 }
```

Ütemezési probléma

Bemenet:

$M = \{m_1, \ldots, m_n\}$ munkák halmaza

$m[i].idotartam > 0$ egész

$m[i].hatarido > 0$ egész

$m[i].haszon > 0$ egész

Kimenet:

$H \subseteq 1..n$

1. A H-beli munkák beoszthatók határidőt nem sértő módon.
2. $$\overline{Cf}(H) = \sum_{i \in H} m[i].haszon \rightarrow maxi \tag{1}$$

H elemeinek egy $\langle i_1, \ldots, i_k \rangle$ felsorolása határidőt nem sértő, ha $\forall 1 \leq j \leq k$

$$\sum_{u=1}^{j} m[i_u].idotartam \leq m[i_j].hatarido \tag{2}$$

Állítás: H-nak akkor és csak akkor van határidőt nem sértő

```
1  #include<bits/stdc++.h>
2
3  using namespace std;
4  vector<int> Hatarido;
5  vector<int> Haszon;
6  vector<int> Vido;
7
8  class FaPont{
9    //Globalis: Hatarido, Haszon, Vido
10   int oido=0;      //a beválasztott munkák összideje
11   int ehaszon=0;  //az elmaradt haszon
12   int maradt;      //a még választható munkák haszna
13   vector<int> A;  //megoldáskezdemény
14   public:
15   bool Elso();
16   bool Testver();
17
18   FaPont(){//konstruktor
19     oido=0; ehaszon=0;  maradt=0;
20     for(int x:Haszon) maradt+=x;
21   }
```

```
22      //
23      int Cf(){return ehaszon+maradt;}//a célfüggvény
24      int AK(){return ehaszon;}
25      int FK(){return ehaszon+maradt;}
26      bool Lehet(){
27        return A.size()==0 || oido<=Hatarido[A[A.size()
28      }
29      bool Megoldas(){
30        return A.size()==0 || oido<=Hatarido[A[A.size()
31      }
32      void Kilr(){
33        if(A.size()==0){cout<<"Mincs␣megoldás"; return;
34        for(int x:A) cout<<x+1<<",";  cout<<endl;
35      }
36    };
```

```
358 struct Rendezo{//a prioritási sor rendezési reláció
359   Rendezo(){};
360   bool operator() (FaPont& X, FaPont& Y) const{
361     return X.FK()>Y.FK();
362   }
363 };
364 //Megoldás keresés felső korlát szerinti
365 //minimumos prioritási sorral erős felső korláttal
```

```
366 FaPont BBKeres2(FaPont X){
367   int G_F_K=INT_MAX;
368   priority_queue<FaPont, vector<FaPont>,Rendezo> S;
369   FaPont M;//az optimális megoldás
370   if (!X.Lehet()) return M;
371   S.push(X);
372   while (S.size()>0 ){
373     X=FaPont(S.top()); S.pop();
374     if (G_F_K<X.AK()) continue; //X kizárt lett
375     if(!X.Elso()) continue;    //X kizárt lett
376     do{
377       if (X.Lehet() && X.AK()<G_F_K ){
378         if (X.Megoldas() && X.Cf()<=G_F_K){//X jobb
379           G_F_K=X.Cf(); M=X;
380         }else if(X.FK()<G_F_K)  G_F_K=X.FK();
381         S.push(X);
382       }//else X kizárt lesz
383     }while(X.Testver());
384   }//while
385   return M;
386 }
```

```
387 bool FaPont::Elso(){
388   if(A.size()==0){
389     A.push_back(0);
390     maradt-=Haszon[0];
391     oido+=Vido[0];
392     return true;
393   }
394   int x=A[A.size()-1];
395   if(x<Haszon.size()-1){
396     A.push_back(x+1);
397     maradt-=Haszon[x+1];
398     oido+=Vido[x+1];
399     return true;
400   }else
401   return false;
402 }
403 bool FaPont::Testver(){
404   if(A.size()==0) return false;
405   int x=A[A.size()-1];
406   if(x<Haszon.size()-1){
407     ehaszon+=Haszon[x];
```

```
408        oido-=Vido[x];  oido+=Vido[x+1];
409        maradt-=Haszon[x+1];
410        A[A.size()-1]++;
411        return true;
412     }else{
413        return false;
414     }
415  }
416
417  int main(){
418     Hatarido={1,2,2,6,9};//határidő
419     Vido={1,6,9,2,1};//vidő
420     Haszon={9,6,3,2,7};//haszon
421     FaPont X=FaPont();//X a megoldástér gyökere
422     FaPont M=BBKeres2(X);
423    M.Kilr();
424  }
```

Feladat 2: Mester: Visszalépéses algoritmusok/ Közbeszerzés.

Valahol országban egy nagyszabású állami beruházás megvalósításán dolgoznak. A beruházás terve M részfeladat elvégzését írja elő. Közbeszerzési pályázatot írtak ki, amelyre V vállalkozó jelentkezett, mindegyik megadta, hogy mely munkát tudja elvégezni, és azt milyen költséggel. Ebben az országban nagy gondot fordítanak a korrupció elkerülésére, ezért olyan vállalkozókat akarnak kiválasztani a pályázók közül, hogy az összköltség a lehető legkisebb legyen. Mivel a munkákat párhuzamosan kell elvégezni, ezért minden pályázó csak egy feladatra kaphat megbízást. Készíts programot, amely kiszámítja, hogy az egyes feladatok elvégzésére melyik pályázót bízzák meg!

Bemenet

Bemenet

A standard bemenet első sorában van a feladatok M száma és a pályázók V száma. A fel-adatokat 1-től M-ig, a vállalkozókat 1-től V-ig sorszámukkal azonosítjuk. A következő M sor mindegyike V darab egész számot tartalmaz. Az i-edik sorban az j-edik szám azt adja meg, hogy az i-edik feladatot a j-edik pályázó mekkora költséggel vállalná el. Ha ez a szám 0, az azt jelenti, hogy a vállalkozó ezt a feladatot nem vállalja el.

Kimenet

A standard kimenet első és egyetlen sorába a lehető legkisebb összköltséget kell írni. A második sor M egész számot tartalmazzon, az i-edik szám annak a vállalkozónak a sorszáma le-gyen, aki az i-edik feladatot fogja elvégezni! Ha nincs a feltételeknek megfelelő megoldás, akkor az első és egyetlen sorba a 0 számot kell kiírni! Több megoldás esetén bármelyik megadható.

Példa

Bemenet	Kimenet
4 7 2 3 0 7 2 3 0 0 2 0 0 8 1 0 9 1 3 0 0 7 5 3 1 1 0 0 8 8	5 5 6 2 3

Korlátok

$1 < M \leq 16$, $1 \leq M \leq V \leq 20$

Időlimit: 2.0 másodperc

Memórialimit: 512 MB.

Pontozás

A pontok 20%-át lehet szerezni olyan bemenetekre, ahol $V \leq 10$.
A pontok további 30%-át lehet szerezni olyan bemenetekre, ahol $V \leq 15$.
A pontok további 50%-át lehet szerezni olyan bemenetekre, ahol nincs egyéb korlátozás.

Gyakorló feladatok

Mester/Visszalépéses keresés: 4. Megye
Mester/Visszalépéses keresés: 22. Munkásfelvétel
Mester/Visszalépéses keresés: 18. L üzlet K pékség